

面试押题：FlashInfer CUDA Graph 挂死根因 + R3 Router Replay

面向：GPU Kernel / 推理引擎 / RL Infra 方向技术面。

口径原则：只讲做过的、能对上代码和实验数据的。边界主动交代，不吹。

关联文档：面试复盘_R3 与 FlashInfer 根因_20260818.md（已有），本文为可直接背诵的压缩版。

§0 两句话定位 + 红线

- **FlashInfer**: TP=2 + FULL CUDA Graph 下 rollout 卡死。根因：fused one-shot allreduce 的 Lamport -0.0 哨兵被 FTZ 误判 → spin-poll 永不退出 → kernel 挂死。修复 = 位级精确比较 `0x80000000`，在我们的 FlashInfer 版本落地 + 两卡最小复现 + 模型级 Graph on/off 回归全通过。
- **R3 Router Replay**: 复现小米论文，解决 vLLM rollout 与 Megatron 训练 MoE 路由不一致导致 logprob drift 破坏 PPO/GRPO 稳定性。打通 `[token, layer, top-k]` route 采集/传输/回放链路 + 建立 response-mask 对齐与异常检测；Qwen3-30B-A3B 8×H200 实测 f_tau_2 降低 36–145×、KL 降低 4–7×；扩展至 128 卡集群，接入 SwanLab 监控。

红线（必须主动交代）

- FlashInfer FTZ 修复是上游/社区已有修法方向，我在我们这个 FlashInfer 版本落地+验证。不要让人误以为是自己发明的算法级修复。

- 简历上 "route mismatch 17–19% → 0" 这句是两套口径缝在一起写的，逻辑上错，读过论文的面试官一问即穿。正确口径：R2/R3 自然路由 mismatch 稳定在 17–19%（与论文同量级，改不了），R3 真正的指标是 **logprob drift 降低**（f_tau_2 降 36–145×、KL 降 4–7×）。面试口述必须用正确口径。

- FlashInfer 和 image v0.4 TP4 那次 hang 不是同一个 bug。本文说的是 MNNVL 文件 `trtllm_allreduce_fusion.cuh` 的 FTZ sentinel bug（TP2 + FULL）；v0.4 TP4 那次是

trtllm_allreduce_fusion.cuh 里 barrier 的 launch-config 发散（不同 bug，不同文件不同机制）。两个别搞混。

主题 A: FlashInfer fused AllReduce + RMSNorm 的 rollout hang 根因

A1. 现象定位路径（下游 → 根因）

条件: VLLM_COMPILE + FULL CUDA Graph + TP=2, H200, Qwen3-30B-A3B BF16。

```
rollout sample_tokens timeout + EngineDeadError
→ 两个 TP rank 在 sampler 之前就阻塞 (sampler 前 synchronize 也 hang)
→ GPU work 已 launch 但 kernel 永不结束
→ 单变量隔离: 只有关掉 fuse_allreduce_rms pass 才 PASS
```

关键单变量控制表:

ID	单变量	结果	结论
R0	compile + graph=NONE + TP=2	PASS 44.63s	compile 本身健康
R1	compile + FULL + TP=2	HANG	最小复现
P1	sampler 前 synchronize()	HANG	replay 在 sampler 前就没结束
P4	blocking D2H copy	HANG	async copy 不是根因
R5	关 async scheduling	HANG	output queue 不是根因
R6	关普通 custom all-reduce	HANG	普通 all-reduce 不是根因
F0	关 fuse_allreduce_rms	PASS 9.18s	单变量隔离到 fused pass

ID	单变量	结果	结论
----	-----	----	----

FIX 配置阶段自动 guard PASS 9.36s 最小修复验证通过

sampler / D2H copy / output queue / MQ timeout 全是下游等待点，不是首因。

A2. 为什么必须 $TP \geq 2$ 才能复现（重要易错点）

- $TP=1$ 没有跨卡规约 $\rightarrow$ fused allreduce+RMSNorm pass 不匹配/不生成 $\rightarrow$ 病根 kernel 根本不出现。
- 最小复现规模就是 2 卡（不是"H200 单卡 141GB 塞不下两份分片"——单卡塞得下，但那条代码路径根本不走）。

A3. Lamport 哨兵——是什么、为什么用

FlashInfer fused one-shot allreduce（低延迟，Hopper 优化）用 **Lamport 算法** 免除显式 barrier:

1. 每个 rank 先把接收 buffer 预置成哨兵值 (sentinel) = "数据还没到"
2. rank A 把数据跨卡 P2P 写进 rank B 的 buffer
3. rank B 不停 spin-poll 自己的 buffer:
 - 还是哨兵值 $\rightarrow$ 数据未到 $\rightarrow$ 继续等
 - 值变了 $\rightarrow$ 数据到了 $\rightarrow$ 读出来参与求和

核心：用"值变了没有"代替"barrier/信号来了没有"——无 barrier，超低延迟。

哨兵值 = **-0.0**，位模式 **0x80000000**（符号位=1，其余全 0）。

选它因为：真实规约数据几乎不可能恰好是它；数值上等于 0，不污染求和；有唯一位模式。

代码对应 (**trtllm_allreduce_fusion.cuh:1376-1393**) :

```
template <>
struct neg_zero<float> {
    static constexpr unsigned int neg_zero_bits = 0x80000000U;
```

```

static constexpr float value = -0.0f;
};

// 修复后的位级精确比较（不是浮点 < 0）：
template <>
__device__ bool is_negative_zero<float>(float x) {
    return (__float_as_int(x) == 0x80000000); // ← 精确比 bit pattern
}

```

A4. FTZ bug——根因核心

次正规数 (subnormal): FP32 最小正规数约 $1.2e-38$; 比它更小但非零的 (如 $1e-40$) 是次正规数。

FTZ (Flush-To-Zero): GPU 浮点模式, 把次正规数直接冲刷成 0。关键: 把负次正规数冲成 **-0.0** (保留符号位), 不是 **+0.0**。

原实现的错误: 轮询用浮点比较识别哨兵:

```

// 原实现 (错):
if (v < 0.0f) → "还是负数 → 还是哨兵 → 继续等"

```

假设"哨兵 -0.0 是负、真实数据非负"。

死循环路径:

1. 规约结果里某个值 = 合法的负次正规数, 如 $-1e-40$
2. 对方 GPU 在 FTZ 下读它 → flush 成 -0.0 (位模式与哨兵完全相同)
3. 轮询: "-0.0 < 0 吗?" → -0.0 在 IEEE754 里 = 0.0, 不 < 0 → ?

实际上原实现用 is_negative 判断, -0.0 的问题是它满足"符号位=1"

但浮点比较 $v < 0$ 时 $-0.0 = 0.0$ 不会 < 0

→ 取决于具体判断方式:

若用 bit check (`bits & 0x80000000`) 判断"负号位" → -0.0 被认为是哨兵 → 永远等

若用 float 比较 < 0 → -0.0 不 < 0 → 认为数据到了 → 结果错

实际 bug: 规约结果的某一元素是负次正规数 → FTZ 冲成 -0.0 →

和预置哨兵 -0.0 bit-identical → poll 误判"数据没到" → 无限自旋

更准确的因果链 (以代码实际行为为准):

1. 某位置的真实规约结果是负次正规数（如 $-1e-40$ ）
2. P2P 写入后，接收方 FTZ 模式下读该值 → 冲刷成 -0.0 ($0x80000000$)
3. `is_negative_zero(x)` 检查 `bit pattern == 0x80000000` → TRUE → "哨兵！数据未到！"
↑ 但数据其实已经到了
4. → spin-poll 永远不退出 → kernel 挂死 → rank 永远出不了 allreduce
→ 另一个 rank + EngineCore 干等 → `sample_tokens timeout` → rollout hang

易错点：hang $\neq$ slow。nsys 里看到的是"**kernel launch 了但没有 end 时间戳**"——不是一个很慢的 kernel，是一个永不结束的 **kernel**。"算子耗时长导致超时"因果是错的。

A5. 修复：位级精确比较

```
// 错：用浮点语义（数值范围）判断哨兵
if (v < 0) → 把所有负数当哨兵，错

// 对：用位级精确比较（唯一位模式）
if (__float_as_int(x) == 0x80000000) → 只认唯一的 bit pattern
```

哨兵是一个**位模式**，不是一个"数值区间"。"是负数" $\supsetneq$ "是 -0.0 "。

A6. 一段话口述版（面试背）

TP ≥ 2 时会走 FlashInfer 的 fused one-shot allreduce+RMSNorm，它用 Lamport 算法免 barrier：接收 buffer 预置成 -0.0 哨兵（bit pattern $0x80000000$ ），轮询到"值变了"就认为数据到了。H200 在 FTZ 模式下，会把合法的负次正规数冲刷成 -0.0 ——位模式和哨兵完全一样。于是当规约结果里出现一个负次正规数时，接收方读到的是 -0.0 ，`is_negative_zero` 返回 true，误判为"还是哨兵、数据没到"，自旋永远退不出，kernel 挂死，那个 rank 出不了 sampler，另一个 rank 和 EngineCore 干等到 timeout，表现成 rollout hang。修复是把哨兵判断从浮点比较改成位级精确比 $0x80000000$ ——因为哨兵是唯一位模式，不是数值区间。我的贡献：定位到 FTZ 误判根因 + 在我们的 FlashInfer 版本上落地位级比较修复 + 两卡最小复现和模型级 Graph on/off 回归全部通过，重复 Graph replay 未再出现 timeout。

A7. 追问预案

Q: FTZ 是什么，为什么 GPU 开着它？

A: Flush-To-Zero，把次正规数 ($|x| < \sim 1.2e-38$) 直接置零。次正规数在硬件上需要特殊处理 (denormalized slow path)，开 FTZ 能提升大量计算的吞吐。代价是损失了 IEEE754 最小精度段的精确度。CUDA 里 `-use_fast_math` 会隐式开它；很多 kernel 为了性能也会手动设 `FTZ=1`。

Q: 为什么负次正规数不是 +0.0 而是 -0.0？

A: IEEE754 规定 FTZ 保留符号位，所以 `-1e-40` $\rightarrow$ `-0.0` (0x80000000) 而不是 `+0.0` (0x00000000)。如果 flush 成 `+0.0` 就不会撞上哨兵了，bug 就消失了——就是因为符号位被保留才出的问题。

Q: 为什么选 -0.0 做哨兵而不是 NaN 或 inf？

A: NaN 在求和时会污染结果 (`x + NaN = NaN`)；inf 同理会溢出。-0.0 数值上等于 0 所以对求和无污染，又有唯一位模式。设计上是合理的，只是没考虑到 FTZ 会把别的值也 flush 成它。

Q: nsys 怎么看出来是 hang 不是 slow？

A: nsys 的 kernel timeline 里，slow kernel 有完整的 start+end 时间条；hang 的 kernel 只有 start，没有 end——timeline 条一直延伸到 profiling 结束为止，或者直接 profiler 本身也超时断开。一眼就看出来是“进去出不来”，不是“很慢但最终结束”。

主题 B: R3 Router Replay

B1. 一句话定义（记 id，不记分数）

推理时 (vLLM) 记录每个 token、每层选中的 **top-k 专家 id** (`routed_experts`，形状 `[token, layer, top-k]`，只记 id，不记分数)；训练时 (Megatron) 跳过自己的 top-k、强制走这组记录的专家，但用训练侧现算的 router 分数 `s_train` 做 gating softmax，然后正常前向、反向、更新。

"选哪些专家"用推理记录钉死；"给这些专家打几分、专家本身好不好"仍由训练侧学习。

为什么只记 id 不记分数（任选一条答都加分）：

1. 省传输/存储：id 是 `[tok, layer, k]`，分数要 `[tok, layer, 专家总数]`，大一个数量级。
2. 保留训练侧梯度：直接用推理记录的分，router 被冻死，学不了；固定"选谁"、分数用 `s_train` 现算，梯度能回到 router。
3. 精准打病根：训推不一致的放大器是离散 **top-k 换专家**，不是分数的小数点漂移。

B2. RL 训练闭环（R3 落在哪）

一个 RL step

① ROLLOUT (vLLM) — 逐 token 自回归采样（慢，FP8/fused，无梯度）

★ R3 多记：routed_experts [token, layer, top-k] = I_infer

|

▼

② REWARD — 对 response 打分（判题/RM） — 与 R3 无关

|

▼

③ OLD_LOG_PROB (Megatron) ★ 纯前向，无反向 ★

整条序列并行 forward → old_log_prob (= π_{train} , ★在计算图里★)

★ R3 作用点①：不自己 top-k，回放 I_infer

【f_tau_2 / KL / k3_kl 在这里量出来】

importance ratio $r = \exp(\text{old_log_prob} - \text{rollout_log_prob}) = \pi_{\text{train}} / \pi_{\text{infer}}$

|

▼

④ ACTOR UPDATE (Megatron) ★ 有反向 ★

PP0/GRPO loss(r, advantage) → backward → optimizer

★ R3 作用点②：这次 forward 也走 I_infer

|

▼

⑤ 权重同步：Megatron → vLLM → 回到 ①

B3. 为什么要两次前向 (③ 不能省)

	① rollout 前向	③ old_log_prob 前向
引擎	vLLM	Megatron
方式	逐 token 采样 (慢)	整条并行 teacher-forcing (快)
产物	π_{infer} (数值)	π_{train} (能 backward 的节点)
带梯度	✗	✓
精度/kernel	FP8 / fused	BF16 / 训练 kernel

- ③ 无法省: ① 无梯度, 拿不去做训练更新; 只有 Megatron 的 forward 才能接着 backward。
- 2× 前向是 PPO/GRPO 的固有成本, 不是 R3 引入的; R3 让 ③ 那次已有的前向走对专家路径, 额外开销 <3% (论文)。
- R3 在两个点起作用, 第一个点没有反向: ③ 纯前向 → drift 指标来源; ④ 反向 → 训练稳定来源。不要说"R3 只在反向时有用"——会被反问"那 f_{tau_2} 哪来的"。

B4. R3 怎么起效果

不用 R3:

- ① vLLM 选 [1,2] (I_{infer})
- ③ Megatron 自己 top-k 选成 [1,3] ← 换了专家!
 - 走完全不同的 FFN 子网络
 - old_log_prob 与 rollout_log_prob 差很大
 - ratio = $\pi_{\text{train}}/\pi_{\text{infer}}$ 出现 0.1 或 10 极端值
 - PPO loss 梯度被放大 → 训练震荡/collapse

用 R3:

- ③ 强制用 [1,2] (回放 I_{infer}) → 走 rollout 真实路径
 - old_log_prob $\approx$ rollout_log_prob → ratio $\approx$ 1 → 梯度干净 → 稳

R3 降的是"后果"不是"路由本身": 路由不一致 (~18%) 客观存在、改不了; R3 让训练侧沿那条已发生的路径重算与更新, 消除 drift 放大效应。

B5. "定死了"还有没有梯度？（关键问题）

定死的和更新的不是同一样东西。

	是什么	R3 定死?	反向更新?
离散选择 I	选哪 top-k 个专家 (id)	☒ 用 I_infer	☒ 本就不可导，无所谓
gating 分数 s_train	router 给每个专家打的分	☒ 训练侧现算	☒ 更新
专家权重 E_i	被选中专家的 FFN 参数	☒	☒ 更新

R3 前向公式（只看被选中的专家 i）：

```
g_replay,i = exp(s_train,i) / Σ_{j∈I_infer} exp(s_train,j)
y_replay    = Σ_{i∈I_infer} g_replay,i · E_i(x_train)
                ↑ I_infer 定死"选谁"          ↑ 两者都是训练侧现算、可导
```

- **I_infer** 只是索引，索引不可导（dense MoE top-k 本来也不可导），但它不需要梯度。
- **g_replay** 用 **s_train** 现算 softmax → 梯度回到 **router 打分参数**。
- **E_i(x_train)** 训练侧真实前向 → 梯度回到**专家权重**。
- **比喻**：定死专家选择 = "这道题必须交给张三李四做"；反向更新 = "他俩做得好不好、router 该多信任他们几分、他俩能力要不要提升"——全照常学。

replay bias：若 actor 更新多次/学习率高，当前 router 想换专家却被 mask 钉住。可接受代价（换来 ratio 可信）；且 ⑤ 每 step 同步权重、重新 rollout，下一轮 **I_infer** 是更新后 router 选的，路径只在**本 step 内固定**，不永久冻结。

B6. mismatch 两套口径——简历冲突的根源

	交付自一致口径（错误用法）	自然路由口径（正确）
开关	<code>metric_use_replay_target_as_actual=True</code> <code>=False</code> ，或单独记训练侧自然 top-k	
拿准比谁	回放 target vs 实际 dispatch	训练侧影子重算自然 I_train vs 推理记录

I_infer

R3 下的值 恒 = 0（定义决定）

~18%，非零，R2/R3 都非零且同量级

证明什么 只证明 routed_experts 被正确交付给 replay 证明训推路由本身有多不一致

偶发非零 采集/传输/对齐 bug（"丢记录"只在此成立） 本来就该非零

- 论文（自然口径）：router 级 ~10% 不一致、token 级 94% 至少一层不同。实测 ~18% 同量级。
- bug 特征（加分讲法）：R2≠0 而 R3=0。
- 最佳口述："我最初用 =True，R3 mismatch=0，以为路由完全对齐。但发现 R2≠0 而 R3=0 的不对称——若 0 是对齐好的结果，R2 不该差这么多。顺着查下去才意识那是自证口径、恒为 0，只能当链路健康检查。于是改口径，R2/R3 回到 ~18% 同量级，R3 真正证据改用 logprob drift，并把这个坑写成团队红线沉淀下来。"

B7. 三句话钉死

1. R3 定死"选哪些专家"（离散、不可导、无需梯度）；反向更新"router 打分 + 专家权重"（可导、照常学）——两者不是一回事。
2. R3 起效点在 ③（纯前向，drift 指标来源）和 ④（反向，训练稳定来源）；"只在反向时有用"是错的。
3. 降的是路由不一致的后果（logprob drift），不是路由不一致本身（~18%，改不了）。R3 只对 MoE 有用，dense 无同类收益。

B8. 追问预案

Q: route mismatch 17%→0 怎么实现的？

A: 没有实现。这是简历口径错误，两套不同口径缝在一起写的。正确答案：自然路由 mismatch 稳定在 17–19%（论文同量级，R3 改不了，也不该改）；R3 真正降的是 logprob drift (f_{τ_2} 36–145×、KL

4-7×)，不是 mismatch 降到 0。（主动说出这个，面试官会加分——体现你真的理解了而不是背数字）

Q：为什么 TP=2 的 fused allreduce 会挂 R3 流程？（两个 topic 联动）

A：R3 跑在 vLLM rollout 里，rollout 是 TP=2 的推理。CUDA Graph capture 时，fused allreduce+RMSNorm 那条路径的 kernel 被 capture 进去；graph replay 时遇到负次正规数被 FTZ 冲成 -0.0，poll 判断误认为哨兵未被替换，kernel 永不退出，rollout sample_tokens timeout，R3 路由记录永远不被发出。所以必须先修 FlashInfer 这个 hang，R3 的 8 卡实验才能稳定跑起来。

Q：128 卡扩展时遇到什么困难？

A：（根据实际经历回答）主要是：(1) 链路对齐问题——128 卡 rollout 里 token 批次和 Megatron 训练侧的 token 批次排列方式不同，response-mask 对齐要处理更复杂的切片逻辑；(2) 传输稳定性——[token, layer, top-k] 的 route 数据在 128 卡下数据量更大，需要保证传输不丢、不错序；(3) 监控可观测性——接入 SwanLab 让 f_tau_2/KL/k3_kl 等 drift 指标实时可视，能快速发现异常（异常时 mismatch 突然为 0、或 drift 突然变大）。

Q：R3 的额外开销是多少？

A：论文报告 <3%。额外开销来自：rollout 侧记录 **routed_experts**（GPU 上已有 top-k 索引，几乎免费）、跨进程传输（相比 response token 数据量小一个数量级）、训练侧 ③ 中替换 top-k dispatch（已经要做的 forward，只是走不同专家路径）。占大头的 rollout 采样和两次 forward 是 PPO/GRPO 固有成本，不是 R3 引入的。

Q：R3 和 DAPO/GRPO 有什么关系？

A：R3 是 **route 一致性** 机制，DAPO/GRPO 是 **训练算法**（loss 设计、reward shaping）。它们正交：R3 可以和任何 PPO/GRPO/DAPO 变体组合用。R3 解决的是"MoE 训推路由不一致导致 importance ratio 失控"这个工程稳定性问题，不改变 loss 形式或 reward 来源。

Q：为什么不直接在 vLLM 里也用 Megatron 的 router forward？

A：vLLM rollout 是 FP8 / fused kernel / 无 autograd 的高吞吐路径，Megatron 的训练

kernel (BF16、带计算图、支持 tensor/expert/pipeline parallel) 不能直接嵌进去——架构和精度都不匹配。加上异构引擎分开部署本来就是 rollout/train 吞吐解耦的需要。R3 记录轻量的 id 传给训练侧，成本极小且不破坏 vLLM 的推理路径。

§C 通用追问（两块都可能问）

Q: TP 和 DP 是什么关系?

A: TP (Tensor Parallel) 切**权重**，一条数据在所有 rank 上都走一遍、最后 all-reduce 合并；DP (Data Parallel) 切**数据**，每个 rank 有完整权重各自前向。"一个卡一份数据"是 DP，不是 TP。TP 才需要 fused allreduce+RMSNorm 这条路径（规约各 rank 的部分和）。

Q: 为什么用 Lamport 而不是 NCCL 做 allreduce?

A: NCCL 走显式 barrier (CPU/GPU 同步点) 或硬件信号，延迟有下界。Lamport one-shot 靠 P2P 写 + 轮询，在低延迟场景（数据量小、rank 数少）能打过 NCCL——这是 FlashInfer fused allreduce 针对 Hopper P2P 带宽优化的设计。代价就是哨兵机制，以及这次 FTZ 引发的 bug。

Q: 整个排查过程最难的是哪步?

A: 难在"hang $\neq$ slow"的认知翻转。最开始把现象描述成"算子耗时长导致超时"，这个因果是错的——实际是 kernel 永不结束，不是 kernel 跑得慢。这个判断靠 nsys 的 kernel timeline 确认 (launch 了但没有 end)，一旦确认是 hang 才锁定到轮询逻辑。